



ESTRUTURA DE DADOS

ESTRUTURA DE DADOS - VETORES, MATRIZES, STRINGS E FUNÇÕES

Prof. Me. Gustavo Ferreira Palma

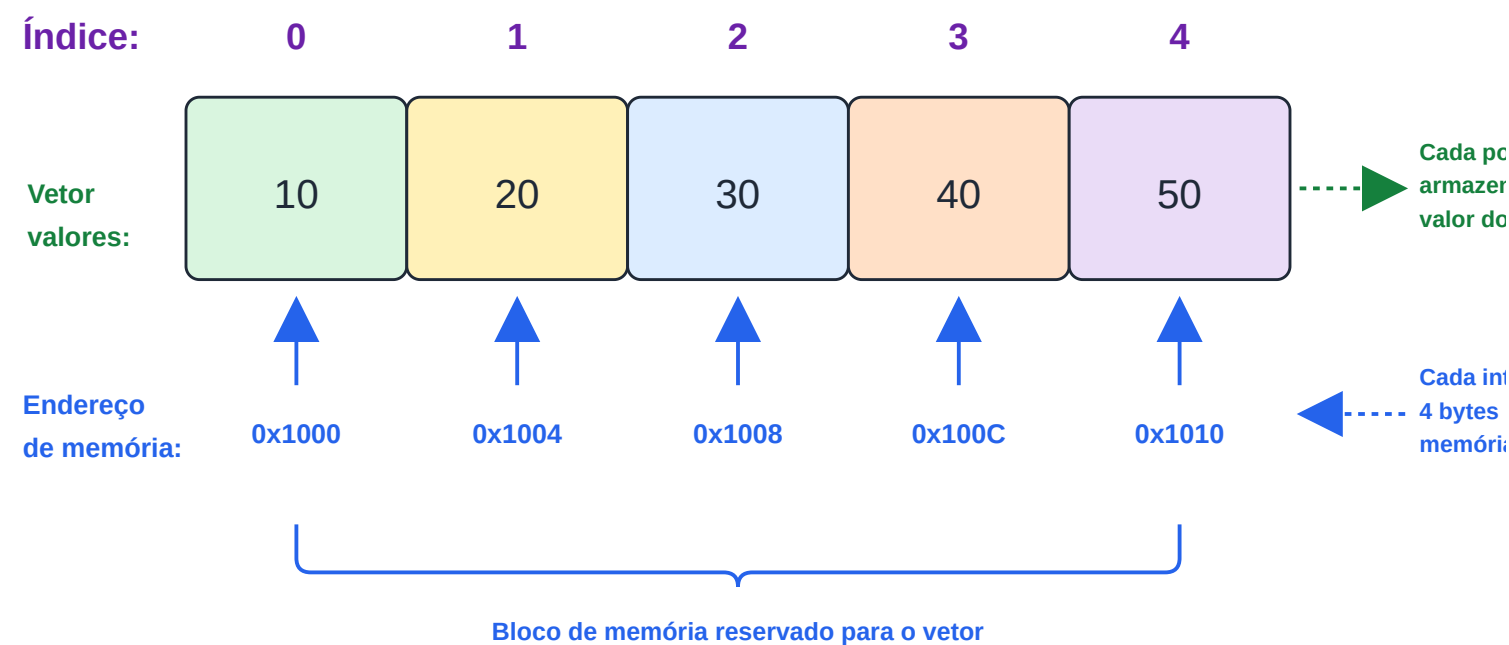
PAUTA

- Vetores
- Matrizes
- Strings
- Funções

VETORES

VETORES - CONCEITO

- Quando tratamos sobre vetores, independentemente da Linguagem de programação escolhida, estamos falando de uma estrutura de dados composta por uma sequência de elementos (valores) do mesmo tipo, **armazenados de forma contínua na memória**. Representando, conceitualmente, uma linha ou uma coluna;
- Cada Elemento de um vetor, é associado a um **índice numérico**. E através desses índices podemos realizar operações de leitura e escrita.



VETORES - DECLARAÇÃO

- A declaração de um vetor informa ao compilador o tipo de seus elementos, o nome da variável e a quantidade de posições que serão reservadas na memória.

"tipo nome_do_vetor[tamanho];"

```
1
2      //...
3      int notas[5];
4      float temperaturas[30];
5      char nome[50];
6      //...
```


VETORES - INICIALIZAÇÃO

- Um vetor pode ser inicializado no momento de sua declaração.

```
1
2 //Inicialização completa:
3 int valores[5] = {10, 20, 30, 40, 50};
4
5 //Inicialização parcial:
6 int numeros[5] = {1, 2};
7 //Nesse caso, as posições restantes serão inicializadas com zero.
8
9 //Também é possível permitir que o compilador determine automaticamente o tamanho do vetor.
10 int numeros[] = {5, 10, 15, 20};
11
12 //Ou ainda inicializar todos os elementos com um valor conhecido
13 int numeros[5] = {0};
```

VETORES - ACESSO ÀS POSIÇÕES

- Cada posição de um vetor é identificada por um índice.
- Na linguagem C, os índices sempre começam em zero.

```
1
2 //escrita
3 scanf("%d", &valores[0]);
4 //escrita
5 valores[2] = 100;
6 //leitura
7 printf("%d", valores[0]);
8 //leitura
9 printf("%d", valores[3]);
```

VETORES - ACESSO ÀS POSIÇÕES

- Ao trabalharmos com vetores, laços de repetição são muito úteis.

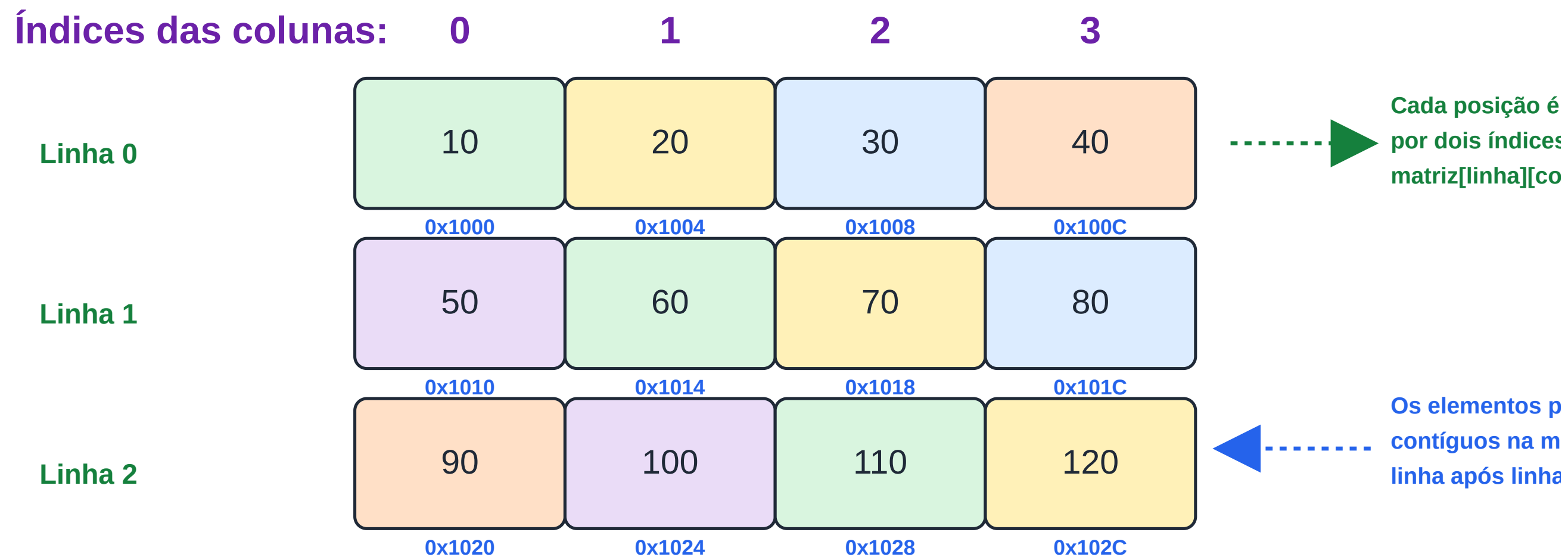
```
1
2 //....
3 #define MAX_VALORES 5
4 int valores[MAX_VALORES] = {0};
5 for (int i = 0; i < MAX_VALORES, i++){
6     if (i == 0)
7         valores[i] = i + 40 + 1;
8     else
9         valores[i] = i + 40;
10 }
11
12 for (int i = 0; i < MAX_VALORES; i++){
13     printf("%c", valores[i]);
14 }
```


MATRIZES

MATRIZES - CONCEITO

- As Matrizes, de forma similar aos Vetores, são estruturas de dados, neste caso bidimensionais, formadas por elementos do mesmo tipo, organizados em linhas e colunas. Seus elementos também **são armazenados de forma contínua na memória;**
- No caso das matrizes, cada elemento é identificado por **dois índices: o primeiro representa a linha e o segundo representa a coluna.**
- As matrizes são amplamente utilizadas para representar tabelas, planilhas, imagens digitais, tabuleiros, mapas e diversas outras estruturas bidimensionais.
- Também existem Matrizes, tridimensionais, que incluem um terceiro índice, que representa a profundidade da matriz

MATRIZES - CONCEITO



Endereço de `matriz[i][j]` = `base + ((i × número de colunas) + j) × sizeof(tipo)`



Bloco contínuo de memória reservado para a matriz

MATRIZES - DECLARAÇÃO

- A declaração de uma matriz informa ao compilador o tipo de seus elementos, o nome da variável e suas dimensões.

tipo nome_da_matriz[linhas][colunas];

```
//...  
int matriz[3][4];  
float notas[10][5];  
char tabuleiro[8][8];  
//...
```

MATRIZES - DECLARAÇÃO

- A declaração de uma matriz tridimensional segue a mesma estrutura, apenas adicionando mais um índice à sua declaração

tipo nome_da_matriz[linhas][colunas][camadas];

```
//...  
//declaração de uma matriz para representar uma imagem digital no padrão RGB  
uint8_t imagem[1920][1080][3];  
/*  
Sendo:  
1920 = largura  
1080 = altura  
3 = Camadas, 0-Red, 1-Green, 2-Blue  
*/  
//...
```


MATRIZES - DECLARAÇÃO

- Uma matriz pode ser inicializada durante sua declaração utilizando listas de inicialização.

```
1
2 //Inicialização completa:
3 int matriz[2][3] =
4 {
5     {1, 2, 3},
6     {4, 5, 6}
7 };
8
9 //Também é possível omitir a primeira dimensão.
10 int matriz[][3] =
11 {
12     {10, 20, 30},
13     {40, 50, 60}
14 };
15
```

MATRIZES - ÍNDICES

- Cada elemento da matriz é identificado por dois índices:
 - o primeiro indica a linha;
 - o segundo indica a coluna.
- Na linguagem C, ambos começam em zero.
- O acesso aos elementos é realizado utilizando dois índices.

```
1
2 //Gravação
3 matriz[1][1] = 100;
4 scanf("%d", &matriz[0][2]);
5 //Leitura
6 printf("%d", matriz[0][2]);
```

MATRIZES - ÍNDICES

- Como uma matriz possui duas dimensões, normalmente utiliza-se um laço de repetição para percorrer as linhas e outro para percorrer as colunas:

```
1
2 #define MAX_LINHA 3
3 #define MAX_COLUNA 3
4 //...
5 for (int i = 0; i < MAX_LINHA; i++)
6 {
7     for (int j = 0; j < MAX_COLUNA; j++)
8     {
9         if (i == 0 && j == 0) {
10             matriz[i][j] = i + j + 40 + 1;
11         } else if (i == 0){
12             matriz[i][j] = i + j + 40;
13         } else {
14             matriz[i][j] = i + j + 40 + 1;
15         }
```


STRINGS

STRINGS - CONCEITO

- Na linguagem C, uma string é um vetor de caracteres utilizado para armazenar textos. Diferentemente de outras linguagens de programação, não existe um tipo específico chamado string; uma string é simplesmente um vetor do tipo char.
- Toda string deve terminar com um caractere especial chamado caractere nulo ('\0'), que indica ao programa onde o texto termina.

STRINGS - DECLARAÇÃO

- Uma string é declarada da mesma forma que um vetor de caracteres.

char nome[tamanho];

```
1
2 //...
3 char nome[50];
4 char cidade[30];
5 char mensagem[100];
6 //...
```

STRINGS - DECLARAÇÃO

- E podem ser inicializados da mesma forma que um vetor

```
1
2 //Uma string pode ser inicializada no momento da declaração.
3 char nome[] = "Gustavo";
4
5 //Também é possível inicializá-la utilizando um vetor de caracteres.
6 char nome[] = {'G', 'u', 's', 't', 'a', 'v', 'o', '\0'};
7 //Ambas as formas produzem exatamente o mesmo resultado.
```

STRINGS - MANIPULAÇÃO

- Sendo as Strings, vetores de caracteres, na linguagem C, sua manipulação, não pode ser feita da mesma forma que manipulamos os tipos primitivos
- A manipulação de Strings é feita através de funções, mais especificamente as funções da biblioteca "string.h"

FUNÇÕES

FUNÇÕES - CONCEITO

- Uma função é um bloco de código responsável por executar uma tarefa específica. Seu principal objetivo é organizar o programa em pequenas partes reutilizáveis, tornando o código mais legível, modular e fácil de manter.
- Ao dividir um problema em funções menores, cada função passa a ter uma única responsabilidade, facilitando tanto o desenvolvimento quanto a manutenção do software.
- O uso de funções é dividido em duas etapas principais: Declaração (prototipagem da função) e a Implementação

FUNÇÕES - CONCEITO

- Na Linguagem C, a função deve ser declarada (prototipada), antes da função main, do contrário teremos um erro de compilação.

```
1
2      //....
3      //prototipagem da função
4      int soma(int a, int b);
5
6      //função main
7      int main() {
8          //...
9          return 0;
10     }
11
12     //Implementação da função
13     int soma(int a, int b) {
14         return a + b;
15     }
```


FUNÇÕES - CONCEITO

- Também é possível declarar e implementar a função antes da função main.

```
1
2      //....
3      //Implementação da função
4      int soma(int a, int b) {
5          return a + b;
6      }
7
8      //função main
9      int main() {
10         //...
11         return 0;
12     }
```

FUNÇÕES - PARÂMETROS

- Os parâmetros representam as informações que uma função recebe para realizar seu processamento.

```
1
2 int multiplicar(int a, int b)
3 {
4     return a * b;
5 }
```

- Nesse exemplo, a e b são parâmetros da função.
- Quando a função é chamada:

```
1
2 int resultado = multiplicar(5, 8);
```

- os valores 5 e 8 são copiados para os parâmetros a e b.
- Na linguagem C, os parâmetros são passados por valor, ou seja, alterações realizadas dentro da função não modificam as variáveis originais.

FUNÇÕES - RETORNO

- Uma função pode retornar um valor para quem a chamou.
- O retorno é realizado utilizando a instrução "return".

```
1
2 float calcularMedia(float n1, float n2)
3 {
4     return (n1 + n2) / 2.0;
5 }
```

- Também existem funções que não retornam nenhum valor. Nesse caso, utiliza-se o tipo void.

```
void limparTela(void)
{
    printf("\033[2J");
}
```

MANIPULANDO STRINGS COM FUNÇÕES

MANIPULANDO STRINGS COM FUNÇÕES

- A biblioteca `string.h` disponibiliza diversas funções para manipulação de strings, como cópia, comparação, concatenação e cálculo do comprimento de um texto.
- Embora existam funções clássicas, como `strcpy()`, `strcmp()` e `strcat()`, nesta disciplina utilizaremos preferencialmente suas versões limitadas (N), que ajudam a evitar erros relacionados ao acesso indevido à memória.

MANIPULANDO STRINGS COM FUNÇÕES - COMPARANDO STRINGS

- A função `strncmp()` compara até uma quantidade máxima de caracteres entre duas strings.

```
1  
2      int strncmp(const char *s1, const char *s2, size_t n);
```

- Onde:
 - `s1` é a primeira string a ser comparada.
 - `s2` é a segunda string a ser comparada.
 - `n` é a quantidade máxima de caracteres considerados na comparação.
 - Retorna `0` quando as strings são iguais.
 - Retorna um valor menor que `0` quando `s1` é menor que `s2`.
 - Retorna um valor maior que `0` quando `s1` é maior que `s2`.

MANIPULANDO STRINGS COM FUNÇÕES - COMPARANDO STRINGS

- Exemplo completo

```
1
2 #include <stdio.h>
3 #include <string.h>
4 int main() {
5     char usuario[] = "admin";
6     if (strncmp(usuario, "admin", sizeof(usuario)) == 0) {
7         printf("Usuário válido.\n");
8     } else {
9         printf("Usuário inválido.\n");
10    }
11    return 0;
12 }
```

MANIPULANDO STRINGS COM FUNÇÕES - COPIANDO STRINGS

- A função `strncpy()` copia uma quantidade máxima de caracteres para outra string.

```
1  
2      char *strncpy(char *destino, const char *origem, size_t n);
```

- Onde:
 - `destino` é a string que receberá a cópia.
 - `origem` é a string que será copiada.
 - `n` é o número máximo de caracteres que poderão ser copiados.
 - Retorna um ponteiro para a string de destino.
 - Quando a string de origem possui tamanho igual ou superior a `n`, o caractere `'\0'` pode não ser copiado, sendo necessária sua inclusão manual.

MANIPULANDO STRINGS COM FUNÇÕES - COMPARANDO STRINGS

- Exemplo completo

```
1
2 #include <stdio.h>
3 #include <string.h>
4 int main() {
5     char origem[] = "Estrutura de Dados";
6     char destino[30];
7     strncpy(destino, origem, sizeof(destino));
8     destino[sizeof(destino) - 1] = '\0';
9     printf("%s\n", destino);
10    return 0;
11 }
```

MANIPULANDO STRINGS COM FUNÇÕES - CONCATENANDO STRINGS

- A função `strncat()` adiciona uma string ao final de outra, respeitando um limite máximo de caracteres.

```
1  
2      char *strncat(char *destino, const char *origem, size_t n);
```

- Onde:
 - `destino` é a string que receberá a concatenação.
 - `origem` é a string que será adicionada ao final da string de destino.
 - `n` é o número máximo de caracteres que poderão ser concatenados.
 - Retorna um ponteiro para a string de destino.
 - É responsabilidade do programador garantir espaço suficiente no vetor de destino para armazenar a concatenação e o caractere `'\0'`.

MANIPULANDO STRINGS COM FUNÇÕES - COMPARANDO STRINGS

- Exemplo completo

```
1
2 #include <stdio.h>
3 #include <string.h>
4 int main() {
5     char mensagem[30] = "Olá ";
6     strncat( mensagem, "Mundo!", sizeof(mensagem) - strlen(mensagem) - 1 );
7     printf("%s\n", mensagem);
8     return 0;
9 }
```

MANIPULANDO STRINGS COM FUNÇÕES

Para referências e exemplos completos acesse [A referência oficial da biblioteca string.h](#)

OBRIGADO!

Podem me encontrar aqui:

- gustavo.palma@unisal.edu.br
- www.gustavopalma.com.br